PENERAPAN RETRIEVAL AUGMENTED GENERATION DALAM ASISTEN DIGITAL UNTUK ANALISIS PERUBAHAN PERATURAN

LAPORAN TUGAS AKHIR

Oleh:

Kumara Daffa Prasetya

105220038

FAKULTAS SAINS DAN ILMU KOMPUTER

PROGRAM STUDI ILMU KOMPUTER

UNIVERSITAS PERTAMINA

AGUSTUS 2045

PENERAPAN RETRIEVAL AUGMENTED GENERATION DALAM ASISTEN DIGITAL UNTUK ANALISIS PERUBAHAN PERATURAN

LAPORAN TUGAS AKHIR

Oleh:

Kumara Daffa Prasetya

105220038

FAKULTAS SAINS DAN ILMU KOMPUTER

PROGRAM STUDI ILMU KOMPUTER

UNIVERSITAS PERTAMINA

AGUSTUS 2045

Universitas
Pertamina

LEMBAR PENGESAHAN

Judul Tugas Akhir	:	PENERAPAN RETRIEVAL AUGMENTED GENERATION DALAM ASISTEN DIGITAL UNTUK ANALISIS PERUBAHAN PERATURAN
Nama Mahasiswa	:	Kumara Daffa Prasetya
Nomor Induk Mahasiswa	:	105220038
Program Studi	:	Ilmu Komputer
Fakultas	:	Sains dan Ilmu Komputer
Tanggal Lulus Sidang Tugas Akhir	:	17 Agustus 2045

Jakarta, 17 Agustus 2045

MENGESAHKAN

Pembimbing

Ade Irawan

NIP.116xxx

MENGETAHUI,

Ketua Program Studi

Ade Irawan

NIP. 116xxx

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa Tugas Akhir berjudul PENERAPAN RETRIEVAL AUGMENTED GENERATION DALAM ASISTEN DIGITAL UNTUK ANALISIS PERUBAHAN PERATURAN ini adalah benar-benar merupakan hasil karya saya sendiri dan tidak mengandung materi yang ditulis oleh orang lain kecuali telah dikutip sebagai referensi yang sumbernya telah dituliskan secara jelas sesuai dengan kaidah penulisan karya ilmiah.

Apabila dikemudian hari ditemukan adanya kecurangan dalam karya ini, saya bersedia menerima sanksi dari Universitas Pertamina sesuai dengan peraturan yang berlaku.

Demi pengembangan ilmu pengetahuan, saya menyetujui untuk memberikan kepada Universitas Pertamina hak bebas royalti noneksklusif (*non-exclusive royalty-free right*) atas Tugas Akhir ini beserta perangkat yang ada. Dengan hak bebas royalti noneksklusif ini Universitas Pertamina berhak menyimpan, mengalih media/format-kan, mengelola dalam bentuk pangkatan data (*database*), merawat, dan mempublikasikan Tugas Akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Jakarta, 17 Agustus 2045

– Materai –

Yang membuat pernyataan,

Kumara Daffa Prasetya

ABSTRAK

Kumara Daffa Prasetya. **105220038**. PENERAPAN RETRIEVAL AUGMENTED GENERATION DALAM ASISTEN DIGITAL UNTUK ANALISIS PERUBAHAN PERATURAN.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetuer at, consectetuer sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetuer. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

Kata kunci: pembelajaran mesin, komunikasi, jaringan, pemrosesan sinyal, energi

ABSTRACT

Kumara Daffa Prasetya. **105220038**. The Title of the Final Project.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Keywords: machine learning, communication, network, signal processing, energy

KATA PENGANTAR

Terima kasih sudah menggunakan template Tugas Akhir ini. Telah habis waktu sekian lamanya untuk membuat dan memperbaiki template ini. Oleh karena itu, tolong jangan hilangkan kredit pembuat dan yang memperkaya template ini, sebagai bentuk apresiasi kalian. Doakan saja semoga ini menjadi amal jariah kami semua. Aamiin.

Seperti yang sering tertulis di bagian Kata Pengantar, tidak ada gading yang tak retak, perbaikan dan saran dari pengguna template ini sangat diharapkan.

Jakarta, 17 Agustus 2025

Kumara Daffa Prasetya

DAFTAR ISI

KATA PENGANTAR . . . . . **iii**

DAFTAR ISI . . . . . **iv**

DAFTAR TABEL . . . . . **vi**

DAFTAR GAMBAR . . . . . **vii**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Rumusan Masalah 2

1.3 Batasan Masalah 3

1.4 Tujuan Penelitian 3

1.5 Manfaat Penelitian 4

BAB II TINJAUAN PUSTAKA **6**

2.1 *Chatbot* 6

2.1.1 Klasifikasi dan perkembangan *Chatbot* 6

2.1.2 Komponen *Natural Language Processing (NLP)* 7

2.1.3 Large Language Models (LLM) 7

2.1.4 Retrieval-Augmented Generation (RAG) 7

2.1.5 Evaluasi Chatbot dan RAGAS 8

BAB III METODE PENELITIAN **10**

3.1 Desain Arsitektur Sistem 10

3.2 *Data Ingestion Pipeline* 11

3.3 *Retrieval Pipeline* 12

3.4 Evaluasi Model 12

BAB IV HASIL DAN PEMBAHASAN **14**

4.1 Pengumpulan dan Pengolahan Data 14

4.2 Skenario Pengujian 15
4.3 Hasil Evaluasi 16
4.4 Studi Kasus Kemampuan *Version-Aware* 16
4.4.1 Analisis Isolasi Konteks 17
4.5 Implementasi Sistem 17
4.5.1 Komponen Pembangun Sistem 17
4.5.2 Alur Pemrosesan Retrieval-Augmented Generation (RAG) 18
4.5.3 Logika Filtering Version-Aware 18

BAB V KESIMPULAN DAN SARAN 20
5.1 Kesimpulan 20
5.2 Saran 20

DAFTAR PUSTAKA 22

LAMPIRAN A Kode Sumber Sistem Retrieval-Augmented Generation (RAG) 24
A.1 Pemrosesan Dokumen dan Basis Data Vektor 24
A.2 Layanan *Backend* API RAG 26
A.3 Antarmuka Pengguna Berbasis Baris Perintah 28
A.4 Evaluasi Kinerja Sistem 29

LAMPIRAN B Judul Lampiran 2 30

DAFTAR TABEL

Tabel 4.1 Dataset Dokumen dan Statistik Chunks 14
Tabel 4.2 Tabel Skenario Pengujian Sistem 15
Tabel 4.3 Hasil Evaluasi Performa RAG Version-Aware 16
Tabel 4.4 Perbandingan Respon Chatbot Berdasarkan Pemilihan Versi Dokumen . . . 16

DAFTAR GAMBAR

Gambar 3.1 Arsitektur RAG (Conventional vs Version Aware) 10

Universitas
Pertamina

BAB I
PENDAHULUAN

1.1 Latar Belakang

Seiring berjalan waktu kemajuan Natural Language Processing (NLP), konsep Retrieval-Augmented Generation (RAG) mulai banyak diaplikasikan untuk menjawab permasalahan akses dan interpretasi dokumen teks yang kompleks. Konsep ini pertama kali diperkenalkan oleh Lewis et al. (2023), yang menggabungkan metode retrieval berbasis pencarian semantik dengan generation menggunakan Large Language Model (LLM). Melalui gabungan tersebut hasil sistem RAG dapat memberikan jawaban yang bersumber langsung dari dokumen rujukan, sehingga bisa meningkatkan akurasi dan transparansi dalam jawaban.

Penerapan teknologi tersebut sangat relevan mengingat pengelolaan dokumen regulasi universitas merupakan hal penting dalam tata kelola akademik dan administratif. Dokumen seperti peraturan rektor, pedoman akademik maupun kebijakan fakultas bersifat dinamis dan terus ada perubahan seiring dengan perkembangan kebijakan pendidikan tinggi. Perubahan tersebut bisa berupa revisi parsial, pembaruan pasal tertentu saja atau penggatian keseluruhan dokumen. Namun, permasalahan sistem pencarian dokumen regulasi yang umum digunakan berbasis kata kunci (keyword-based retrieval), sehingga sering terjadi gagal membedakan versi dokumen masih berlaku dan tidak bisa menelusuri hubungan antarversi regulasi yang saling terkait. Hal ini menimbulkan kesulitan bagi civitas akademik dalam memastikan validitas dan relevansi suatu kebijakan universitas.

Penelitian yang sudah ada menunjukan bahwa RAG memiliki kemampuan adaptif di berbagai ranah. Mengimplementasikan RAG pada bidang pendidikan teknik (*ChAx*) untuk membantu mahasiswa memahami dokumen teknis yang kompleks Steininger, dkk. (2025). Sementara itu, Amazou, dkk. (2025) menerapkannya dalam konteks hukum kontrak untuk meningkatkan akurasi sitasi. Melengkapi temuan tersebut pada domain kritis yang membutuhkan validitas tinggi, membuktikan bahwa integrasi RAG pada sistem kesehatan mampu mengurangi risiko halusinasi model bahasa secara signifikan dan meningkatkan relevansi jawaban berbasis bukti Samudra, dkk. (2025). Karakteristik akurasi tinggi ini sangat krusial untuk diadopsi ke dalam sistem pencarian regulasi universitas guna meminimalisir kesalahan interpretasi administratif.

Meskipun memberikan hasil menjanjikan, penelitian-penelitian tersebut menyoroti tantangan teknis yang belum sepenuhnya terpecahkan, terutama terkait struktur dokumen dan keamanan data. Steininger, dkk. (2025) menemukan bahwa metode pemecahan dokumen (*chunking*) standar sering kali memutus konteks hirarki (bab dan sub-bab), yang fatal bagi dokumen dengan ketergantungan aturan yang tinggi seperti gambar teknik atau regulasi. Selain itu, mengingat sifat dokumen internal universitas yang sensitif, aspek keamanan data menjadi perhatian utama. Anjaneyulu, dkk. (2025) menyarankan implementasi sistem RAG yang beroperasi secara *offline* untuk menjamin privasi dokumen institusi agar tidak terekspos ke layanan *cloud* publik. Namun, kebanyakan masih berfokus pada dokumen statis dan belum menangani hubungan versi atau konteks temporal yang dinamis se-

cara mendalam. Dalam konteks dokumen regulasi universitas, pendekatan RAG konvensional menghadapi masalah tambahan karena sifat dokumen yang amandatif (dapat berubah sebagian) dan temporal (memiliki periode berlaku). Kompleksitas ini memiliki permasalahan yang perlu diselesaikan dalam pengembangan asisten digital berbasis RAG untuk regulasi universitas, yaitu:

1. **Temporal Inconsistency**
   Sistem tidak dapat membedakan konteks waktu dari informasi, sehingga berpotensi memberikan jawaban berdasarkan dokumen yang sudah tidak berlaku.

2. **Partial Supersession**
   Perubahan regulasi di universitas sifat parsial, dimana hanya beberapa pasal atau ayat yang diperbarui. Sistem RAG tanpa mekanisme versi cenderung mencampur dokumen lama dan baru.

3. **Citation Accuracy**
   Ketika sistem tidak memiliki manajemen versi dengan baik, referensi terhadap sumber menjadi tidak akurat. Hal ini bisa menimbulkan resiko kesalahan interpretasi terhadap kebijakan yang berlaku.

4. **Structural Semantics Loss**
   Dokumen regulasi memiliki struktur yang kompleks jika proses *chunking* dan *embedding* tidak mempertimbangkan struktur sistem akan kehilangan konteks dan mempengaruhi dalam pencarian dokumen yang relevan.

1.2 Rumusan Masalah

Berdasarkan latar belakang permasalahan yang telah diuraikan, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana menerapkan metode *chunking* dan *embedding* yang efektif untuk mempertahankan struktur hirarki dokumen regulasi (bab, pasal, ayat) agar konteks tidak hilang saat proses *retrieval*?

2. Bagaimana mekanisme *retrieval* dan *generation* dapat dioptimalkan untuk memastikan akurasi sitasi, sehingga jawaban yang dihasilkan sistem merujuk secara tepat pada dokumen yang relevan?

3. Sejauh mana peningkatan kinerja sistem RAG dengan pendekatan *version-aware* dibandingkan RAG konvensional dalam aspek akurasi jawaban dan relevansi dokumen regulasi universitas?

1.3 Batasan Masalah

Agar penelitian ini lebih terarah dan fokus pada tujuan yang ingin dicapai, penulis menetapkan beberapa batasan masalah sebagai berikut:

1. **Lingkup Data**: Data yang digunakan dalam penelitian ini terbatas pada dokumen regulasi internal Universitas Pertamina (seperti Peraturan Rektor, SK, dan Pedoman Akademik) dalam format teks digital (PDF).
2. **Model Bahasa**: Penelitian ini menggunakan *Pre-trained Large Language Model* (LLM) sebagai generator jawaban (misalnya model berbasis GPT atau Llama) dan tidak melakukan proses *fine-tuning* atau pelatihan ulang pada model tersebut.
3. **Fokus Versioning**: Kemampuan *version-aware* pada sistem dibatasi pada identifikasi metadata dokumen (seperti tanggal berlaku dan nomor surat) untuk memfilter dokumen relevan, bukan melakukan analisis interpretasi hukum mendalam terhadap konflik antar pasal.
4. **Bahasa**: Sistem hanya dirancang untuk memproses dan menghasilkan jawaban dalam Bahasa Indonesia.
5. **Input Pengguna**: Pertanyaan yang diajukan pengguna diasumsikan berupa pertanyaan terkait kebijakan universitas, bukan pertanyaan yang bersifat opini atau prediksi masa depan.

1.4 Tujuan Penelitian

Tujuan utama dari penelitian ini adalah mengembangkan sistem pencarian informasi berbasis *Retrieval-Augmented Generation* (RAG) yang mampu menangani perubahan regulasi universitas. Secara spesifik, tujuan penelitian ini adalah:

1. Mengimplementasikan strategi *chunking* dan *embedding* yang mampu mempreservasi struktur hirarki dokumen (bab, pasal, ayat) untuk menjaga konteks semantik yang utuh.
2. Merancang mekanisme sitasi otomatis dalam proses *generation* untuk memastikan setiap jawaban yang diberikan memiliki rujukan ke dokumen regulasi yang digunakan.
3. Mengevaluasi performa sistem RAG yang dikembangkan menggunakan metrik relevansi dan akurasi (seperti *Precision*, *Recall*, atau *RAGAS score*) dibandingkan dengan pendekatan pencarian konvensional.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1. Membantu Sivitas Akademika Universitas Pertamina dalam mencari informasi regulasi secara akurat, khususnya dalam memastikan dokumen yang ditemukan adalah versi yang berlaku dan valid.
2. Meningkatkan efisiensi pencarian dokumen hukum universitas melalui teknologi *chatbot* berbasis *Retrieval-Augmented Generation* (RAG) yang mampu memahami konteks pertanyaan pengguna dengan lebih baik dibandingkan pencarian kata kunci biasa.
3. Menambah wawasan dan menjadi referensi bagi peneliti lain dalam pengembangan sistem tanya jawab otomatis (*chatbot*) berbasis teknologi *Retrieval-Augmented Generation* (RAG).

Universitas
Pertamina

BAB II
TINJAUAN PUSTAKA

2.1 *Chatbot*

Penggunaan *Chatbot* berkembang cukup pesat diberbagai bidang apapun dalam beberapa tahun terakhir seperti pemasaran, sistem pendukung, pendidikan, perawatan kesehatan sampai hiburan. Perkembangan ini didukung dengan kemajuan teknologi kecerdasan buatan yang mampu beradaptasi layaknya komunikasi dengan manusia. Secara definisi, (*Chatbot*) adalah program komputer yang dirancang untuk mensimulasikan diskusi dengan pengguna manusia, terutama melalui internet. Dalam literatur, teknologi ini juga dikenal dengan sebutan seperti (*smart bots*), agen interaktif, asisten digital, atau entitas percakapan buatan (*artifical conversation entities*). Tujuan utama dari sistem ini tidak hanya semata untuk meniru percakapan manusia sehari-hari, tetapi juga bisa beguna untuk membantu kegiatan manusia dibidang lain.

2.1.1 Klasifikasi dan perkembangan *Chatbot*

Seiring dengan perkembangan, *chatbot* dapat diklasifikasikan berdasarkan metode pemprosesan input dan cara memberikan hasil respon. terdapat tiga model yang digunakan yaitu:

1. **Rule-based model**
   Merupakan arsitektur yang digunakan oleh sebagaian besar *chatbot* pada generasi awal. Pada sistem ini memiliki respon berdasarkan kumpulan aturan yang ditetapkan dan mengenali bentuk kosakata dari teks input tanpa membuat jawaban baru. Pengetahuan pada *chatbot* ini buat secara manual *hand-coded*, sehingga model ini memiliki kelemahan seperti kesalahan ejaan atau tata bahasa dalam input pengguna.

2. **Retrieval-based model**
   Pada model ini memberikan fleksibilitas karena dapat melakukan *query* dan menganalisis sumber data yang didapat melalui *API*. *Chatbot* berbasis retrieval memberikan respons berdasarkan indeks yang sudah ada, sehingga proses pencocokan respon yang diberikan sesuai.

3. **Generative model**
   Model ini memberikan jawaban dengan cara lebih baik dari model sebelumnya dengan menggunakan algoritma *machine learning* dan *deep learning*. *Chatbot* jenis ini hampir mendekati manusia (*human-like*) karena mampu memberikan respon baru berdasarkan pesan pengguna saat ini dan sebelumnya.

2.1.2 Komponen *Natural Language Processing (NLP)*

Agar *chatbot* dapat berinteraksi secara efektif, sistem ini mengandalkan *NLP*. *NLP* adalah bidang kecerdasan buatan yang memanipulasi teks atau ucapan bahasa alami oleh komputer. Inti tugas *NLP* untuk *Natural Language Understanding (NLU)*, yang merupakan teknik mengimplementasikan respon kepada pengguna secara alami. *NLU* bertujuan untuk memproses konteks dan kata dari input bahasa alami pengguna yang mungkin tidak terstruktur, kemudian merespon sesuai dengan niat pengguna. Dalam proses ini, terdapat dua konsep penting:

I. Intent : Mewakili pemetaan antara apa yang dikatakan pengguna dan tindakan apa yang harus diambil oleh *chatbot*.

II. Entity : Merupakan nilai parameter spesifik dari input bahasa alami, seperti tanggal atau nama lokasi, untuk melengkapi pemahaman sistem terhadap permintaan pengguna.

2.1.3 Large Language Models (LLM)

Large Language Models (LLM) telah mendapatkan banyak perhatian dalam beberapa tahun terakhir dan diterapkan di berbagai industri. LLM didasarkan pada algoritma *deep learning* dan jaringan saraf tiruan (*artificial neural networks*) untuk memahami, menafsirkan, dan menghasilkan jawaban dalam bahasa manusia Steininger, dkk. (2025).

Proses pembuatan dan pelatihan LLM membutuhkan jumlah data yang sangat besar. Proses ini dimulai dengan fase *unsupervised learning* menggunakan data tidak terstruktur, dilanjutkan dengan *fine-tuning* menggunakan data terstruktur untuk konsep yang lebih spesifik, dan diakhiri dengan *reinforcement learning* Steininger, dkk. (2025).

Meskipun canggih, LLM memiliki kelemahan, yaitu sulitnya penyesuaian (*customization*) untuk topik spesifik karena kebutuhan data pelatihan yang besar Steininger, dkk. (2025). Selain itu, keterlacakan (*traceability*) jawaban yang diberikan model menjadi permasalahan, karena tidak selalu jelas bagaimana model memperoleh jawabannya. kondisi dapat menyebabkan halusinasi atau informasi yang salah. Salah satu cara untuk mengatasi masalah ini adalah penerapan *Retrieval-Augmented Generation* (RAG) Steininger, dkk. (2025).

2.1.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) pertama kali diperkenalkan oleh Lewis et al. (2020) sebagai modifikasi untuk LLM yang membatasi referensi data pada sekumpulan informasi tertentu, biasanya dari dokumen spesifik Steininger, dkk. (2025). Pendekatan ini memungkinkan penggunaan model untuk domain khusus.

RAG dibangun sebagai pendekatan dua langkah yang menggabungkan *retriever* (non-parametrik) dengan model *generator* (memori parametrik) Steininger, dkk. (2025).

1. **Retriever:** Terdiri dari *query encoder* yang mengubah pertanyaan pengguna menjadi representasi vektor, serta indeks dokumen. Indeks dokumen menggunakan matriks kesamaan (*similarity matrix*) untuk mengidentifikasi dan mengambil dokumen yang paling sesuai untuk menjawab pertanyaan Steininger, dkk. (2025).

2. **Generator:** Menggunakan dokumen yang diambil dari indeks beserta pertanyaan pengguna untuk membuat respons akhir Steininger, dkk. (2025). Dokumen yang diambil dapat dibatasi jumlahnya menggunakan pendekatan *top-k approximation*, di mana k adalah hiperparameter yang dapat disesuaikan Steininger, dkk. (2025).

2.1.5 Evaluasi Chatbot dan RAGAS

Mengevaluasi kualitas *pipeline* chatbot sangat penting untuk memastikan keberhasilan dan keakuratannya Steininger, dkk. (2025). Metode evaluasi tradisional seperti ROUGE dan BLEU, yang awalnya dikembangkan untuk penerjemahan mesin, mengandalkan *n-grams* untuk membandingkan teks yang dihasilkan dengan referensi Steininger, dkk. (2025).

Pendekatan yang lebih modern adalah menggunakan metode *LLM-as-a-judge*, di mana LLM digunakan untuk menilai output chatbot berdasarkan kriteria tertentu Steininger, dkk. (2025). Salah satu kerangka kerja yang menerapkan metodologi ini adalah RAGAS, yang menawarkan metrik yang kuat untuk mengevaluasi berbagai aspek kinerja RAG Steininger, dkk. (2025). Beberapa metrik utama dalam evaluasi ini meliputi:

- **Factual Correctness:** Mengevaluasi keakuratan faktual dari respons yang dihasilkan dengan membandingkannya dengan referensi dasar Steininger, dkk. (2025).
- **Context Precision:** Mengukur proporsi potongan (*chunks*) yang relevan di dalam konteks yang diambil (*retrieved context*) Steininger, dkk. (2025).
- **Context Recall:** Menilai kelengkapan pengambilan dokumen dengan mengukur seberapa banyak dokumen relevan yang berhasil diambil Steininger, dkk. (2025).
- **Faithfulness:** Mengevaluasi konsistensi faktual dari jawaban yang dihasilkan terhadap konteks yang diberikan, untuk memastikan jawaban tidak mengandung informasi yang tidak didukung (halusinasi) Steininger, dkk. (2025).
- **Answer Relevancy:** Memeriksa seberapa dekat jawaban yang dihasilkan menjawab pertanyaan awal (*prompt*) pengguna Steininger, dkk. (2025).

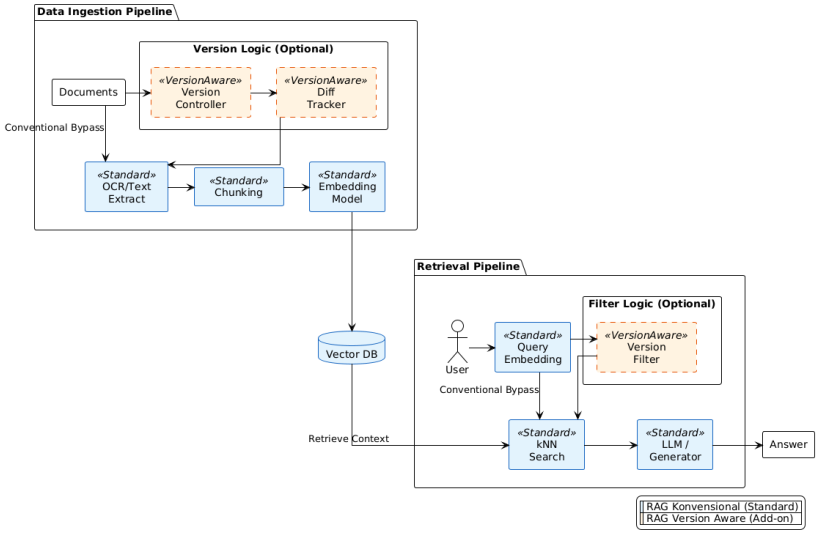
Universitas
Pertamina

BAB III
METODE PENELITIAN

Untuk menyelesaikan permasalahan yang dirumuskan, penelitian ini mengembangkan sistem *chatbot* dengan arsitektur yang mengintegrasikan metode konvensional dengan *version-aware*. Metodologi penelitian dirancang untuk menangani dokumen yang memiliki riwayat perubahan (versi) secara kronologis.

3.1 Desain Arsitektur Sistem

Sistem dibangun berdasarkan arsitektur yang memisahkan antara komponen standar RAG dengan version aware. Gambaran umum arsitektur sistem dapat dilihat pada Gambar 3.1.



Gambar 3.1. Arsitektur RAG (Conventional vs Version Aware)

Seperti yang ditunjukkan pada Gambar 3.1, arsitektur ini terdiri dari dua jalur utama (*pipeline*):

1. ***Data Ingestion Pipeline***: Bertanggung jawab untuk memproses dokumen mentah menjadi representasi vektor yang tersimpan dalam basis data.
2. ***Retrieval Pipeline***: Bertanggung jawab untuk menangani pertanyaan pengguna, melakukan pencarian relevansi, dan menghasilkan jawaban.

Komponen berwarna biru merepresentasikan proses RAG standar (*Standard Component*), sedangkan komponen berwarna oranye dengan garis putus-putus merepresentasikan logika tambahan (*Add-on Logic*) untuk menangani kapabilitas *version-aware*.

3.2 *Data Ingestion Pipeline*

Tahap ini dapat dilihat pada gambar 3.1. Proses ini bertujuan mengubah dokumen tidak terstruktur menjadi format yang dapat dipahami oleh mesin dengan menyertakan konteks versi.

1. ***Version Logic (Controller)***
   Sebelum dokumen diproses secara fisik, *Version Controller* mengidentifikasi metadata dokumen seperti Nomor Dokumen, Tag Versi (misalnya v1, v2), dan *Timestamp*. Pada implementasi kode, logika ini memastikan setiap dokumen yang masuk ditandai dengan ID unik dan waktu unggah untuk membedakan antara dokumen lama dan dokumen terbaru.

2. ***OCR/Text Extract***
   Dokumen PDF diproses menggunakan pustaka *pdfplumber* dengan mode *layout-aware* untuk mempertahankan struktur spasial teks. Jika dokumen berupa citra, sistem menggunakan *pytesseract* (OCR) untuk mengekstraksi karakter visual menjadi teks digital.

3. ***Chunking***
   Penelitian ini menerapkan metode *Line-Aware Chunking*. Berbeda dengan pemotongan karakter statis, metode ini membagi teks berdasarkan baris logis yang telah dibersihkan dari karakter sampah (*noise*). Setiap *chunk* kemudian diasosiasikan dengan metadata versi yang telah disiapkan sebelumnya.

4. ***Embedding Model***
   Teks yang telah terbagi dikonversi menjadi vektor numerik menggunakan model *intfloat/multilingual-e5-base*. Model ini dipilih karena kemampuannya yang optimal dalam menangani teks multibahasa.

5. ***Vector DB***
   Vektor dan metadata disimpan ke dalam *ChromaDB*. Basis data ini bertindak sebagai jembatan penyimpanan antara proses *ingestion* dan *retrieval*.

3.3 *Retrieval Pipeline*

Tahap ini ditunjukkan pada gambar 3.1, yang menjelaskan bagaimana sistem merespons masukan pengguna.

1. ***Query Embedding***
   Pertanyaan pengguna dikonversi menjadi vektor menggunakan model yang sama dengan tahap *ingestion*.

2. ***Version Filter (VersionAware Logic)***
   Ini adalah komponen kunci yang membedakan penelitian ini dengan RAG konvensional. Sebelum atau sesudah pencarian vektor, sistem menerapkan filter logika:

   - Jika pengguna meminta **versi spesifik** (misal: "Gunakan v1"), sistem memfilter metadata `version='v1'`.
   - Jika pengguna meminta **versi terbaru** (*Latest*), sistem melakukan pemfilteran pasca-pencarian (*post-retrieval filtering*) untuk membandingkan *timestamp* antar dokumen yang memiliki ID sama, dan hanya mengambil dokumen dengan waktu terkini.

3. ***kNN Search***
   Sistem melakukan pencarian *k-Nearest Neighbors* (kNN) pada basis data vektor untuk menemukan potongan teks (*chunks*) yang paling mirip secara semantik dengan pertanyaan pengguna, yang telah lolos dari tahapan *Version Filter*.

4. ***LLM / Generator***
   Konteks yang relevan disusun menjadi *prompt* dan dikirim ke *Large Language Model* (Ollama/Llama3). Model diperintahkan untuk menjawab hanya berdasarkan konteks yang ditemukan.

3.4 Evaluasi Model

Untuk mengukur efektivitas arsitektur *Version-Aware* ini dibandingkan dengan RAG Konvensional, maka dilakukan evaluasi menggunakan metrik RAGAS (*Retrieval Augmented Generation Assessment*).

1. ***Context Precision***: Mengukur apakah sistem berhasil mengambil versi dokumen yang tepat (misal: tidak mengambil dokumen v1 saat diminta v2).

2. ***Faithfulness***: Mengukur seberapa akurat jawaban yang dihasilkan berdasarkan konteks versi yang diambil, tanpa halusinasi.

3. ***Answer Relevancy***: Mengukur relevansi jawaban terhadap pertanyaan pengguna.

Universitas
Pertamina

BAB IV
HASIL DAN PEMBAHASAN

Pada tahap ini merupakan proses pengumpulan data yang dipakai untuk uji coba, ada sebanyak 6 dokumen yang digunakan sebagai data untuk *chunk* dengan total 1.324 *chunk* dan dokumen yang sudah chunk disimpan ke dalaman *ChromaDB* sebagai database.

4.1 Pengumpulan dan Pengolahan Data

Tahap awal dalam penelitian ini adalah persiapan data yang menjadi dasar pengetahuan (*knowledge base*) bagi *chatbot*. Total sebanyak 10 dokumen berformat PDF digunakan sebagai data uji. Dokumen-dokumen ini dipilih untuk merepresentasikan [Jelaskan jenis dokumenmu, misal: regulasi yang memiliki versi lama dan baru].

Setiap dokumen melalui proses *preprocessing* dan dibagi menjadi potongan-potongan teks (*chunks*). Proses ini penting agar model dapat memproses informasi dalam batasan *context window* yang tersedia. Rincian statistik data dokumen yang disimpan ke dalam *Vector Database* (ChromaDB) dapat dilihat pada Tabel 4.1.

Tabel 4.1. Dataset Dokumen dan Statistik Chunks

No	Nama Dokumen	Halaman	Jumlah Chunks
1	0166 Kalender Akademik TA 2022-2023	8	47
2	SK - Kalender Akademik TA 2024-2025	9	53
3	234 - 2025 SK - Kalender Akademik 2025-2026	10	44
4	007 - Buku Pedoman Implementasi MBKM	137	918
5	PER - 0013 - 2021 - Peraturan Implemen	41	219
6	Salinan-Kepmen-Nomor-74-P-2021-tentang	6	43
Total Chunks			**1.324**

Dokumen-dokumen yang ada pada tabel ini dipilih karena sesuai dengan kemampuan *chatbot* yang bisa memproses file berformat pdf murni bukan dari gambar atau foto yang menjadi pdf serta relevansi dokumen yang yang terkait topik yang dibahas merupakan peraturan akademik serta standar operasional digunakan di Universitas Pertamina. Dokumen-dokumen yang dipilih mencakup peraturan akademik, proses prosedur standar, dan pedoman lain yang penting bagi sivitas akademika untuk menjalankan tugas. Pemilihan dokumen berdasarkan untuk keperluan informasi yang terkait institusi tersebut, sehingga informasi yang diberikan oleh *chatbot* juga terkait institusi tersebut.

4.2 Skenario Pengujian

Pengujian dilakukan untuk memvalidasi kemampuan sistem dalam dua aspek utama:

1. **Akurasi Retrieval:** Kemampuan sistem mengambil konteks dari versi dokumen yang tepat sesuai permintaan pengguna.
2. **Kualitas Jawaban:** Keakuratan jawaban yang dihasilkan oleh LLM (Llama3:8b) berdasarkan konteks yang diambil.

Mekanisme pengujian dilakukan dengan mengajukan pertanyaan ke *chatbot*, yang kemudian diproses melalui tahapan *embedding*, *retrieving*, dan *generating*. Tabel 4.2 berikut menjabarkan rencana pengujian fungsional sistem, khususnya terkait fitur *Version Aware*.

Tabel 4.2. Tabel Skenario Pengujian Sistem

Kode Uji	Skenario	Ekspektasi Hasil	Status
TC-01	**Default Query (Dokumen Baru)** Mengajukan pertanyaan terkait regulasi terbaru tanpa memilih versi khusus.	Sistem otomatis mengambil konteks dari dokumen versi terbaru (tahun berjalan).	Berhasil
TC-02	**Query Dokumen Lama (Tanpa Filter)** Mengajukan pertanyaan spesifik tentang data lama (misal: Kalender 2022) tanpa mengubah setting versi.	Sistem menjawab ”Informasi tidak ditemukan” atau menyarankan cek versi lain (Sistem tidak mencampuradukkan data).	Berhasil
TC-03	**Version Switching** Mengubah filter ke versi ”v1” (Dokumen Lama) dan mengajukan pertanyaan yang sama.	Sistem berhasil mengambil konteks dari dokumen lama yang sebelumnya tidak terdeteksi.	Berhasil
TC-04	**Relevansi Jawaban** Memastikan jawaban yang digenerate LLM sesuai dengan teks dokumen.	Jawaban tidak berhalusinasi dan menyertakan referensi dokumen yang benar.	Berhasil

Sesuai Tabel 4.2, apabila pertanyaan yang diajukan berhubungan dengan dokumen lama namun pengguna sedang berada di *version* dokumen baru, sistem dirancang untuk tidak memberikan jawaban yang ada pada dokumen lama, memberikan informasi ketiadaan konteks tidak ada pada dokumen. Pengguna diharuskan memilih versi dokumen yang relevan (misalnya ”v1”) agar *chatbot* dapat merespon pertanyaan terkait isi dokumen tersebut.

4.3 Hasil Evaluasi

Berdasarkan tabel 4.3, terlihat sistem menunjukkan performa yang cukup baik pada metrik **Context Precision** dengan skor 0.600. Skor ini menunjukkan bahwa fitur *version aware* yang diimplementasikan pada sistem menggunakan cara *metadata filtering* di ChromaDB bekerja. Sistem berhasil memisahkan dan memilih potongan dokumen *chunks* yang relevan dari versi dokumen yang diminta pengguna dan meminimalisir *noise* dari dokumen lain yang memiliki kesamaan konten.

Nilai **Faithfulness** sebesar 0.733 menunjukkan bahwa halusinasi model bisa diminimalisir, sehingga jawaban yang diberikan oleh LLM juga sesuai dengan konteks yang diambil dari data vektor yang ada, bukan dari jawaban yang dibuat oleh LLM itu sendiri.

Tabel 4.3. Hasil Evaluasi Performa RAG Version-Aware

Metrik Evaluasi	**Skor Rata-rata**
Context Precision	0.600
Context Recall	1.000
Faithfulness	0.733
Answer Relevancy	0.907

4.4 Studi Kasus Kemampuan *Version-Aware*

Bagian ini bertujuan untuk menguji dan membuktikan efektivitas fitur utama sistem, yaitu *metadata filtering* dalam menangani dokumen yang memiliki kemiripan konteks namun berbeda masa berlakunya (*versioning*).

Pengujian kualitatif ini menggunakan dokumen ”Kalender Akademik” Universitas Pertamina. Secara natur, kalender akademik selalu diperbarui setiap tahun, sehingga rentan menyebabkan kebingungan (*ambiguity*) pada sistem RAG tradisional jika tahunnya tidak difilter. Skenario dilakukan dengan mengajukan satu pertanyaan yang sama persis pada dua kondisi pemilihan versi dokumen yang berbeda.

Tabel 4.4 memperlihatkan perbandingan respons *chatbot* saat pengguna menanyakan jadwal Ujian Akhir Semester (UAS).

Tabel 4.4. Perbandingan Respon Chatbot Berdasarkan Pemilihan Versi Dokumen

Pertanyaan	**Filter Versi**	**Respon Chatbot**	**Status**
”Kapan pelaksanaan Ujian Akhir Semester (UAS) Ganjil dimulai?”	**TA 2022-2023**	”Berdasarkan Kalender Akademik TA 2022/2023, pelaksanaan UAS Ganjil dimulai pada tanggal **16 Januari 2023 - 27 Januari 2023**.”	**Valid**
	TA 2024-2025	”Sesuai SK Kalender Akademik TA 2024/2025, Ujian Akhir Semester (UAS) Ganjil akan dilaksanakan mulai **13 Januari 2025 - 25 Januari 2025** .”	**Valid**

4.4.1 Analisis Isolasi Konteks

Berdasarkan hasil pengujian pada Tabel 4.4, dapat dianalisis beberapa keunggulan dari implementasi *version-aware*:

1. **Isolasi Konteks yang Sukses:** Saat pengguna memilih filter "TA 2024-2025", sistem secara otomatis mengabaikan seluruh potongan teks (*chunks*) dari dokumen kalender tahun 2022. Meskipun kedua dokumen mengandung kata kunci "UAS" dan "Ganjil", proses *pre-filtering* di ChromaDB memastikan hanya data tahun 2024 yang ditarik ke dalam *prompt*.
2. **Pencegahan Halusinasi Gabungan:** Tanpa fitur ini, sistem RAG standar berpotensi mencampurkan kedua informasi tersebut (misalnya menjawab: "UAS dilaksanakan pada Desember 2022 dan Januari 2025"), yang justru memberikan informasi keliru kepada pengguna.
3. **Pelacakan Sumber (*Source Tracking*):** Pada setiap jawabannya, LLM (Llama3:8b) secara konsisten menyebutkan referensi tahun keluaran dokumen ("Berdasarkan Kalender Akademik TA 2022/2023..."), yang meningkatkan tingkat kepercayaan pengguna terhadap validitas informasi yang diberikan.

Hasil studi kasus ini mengonfirmasi bahwa penambahan atribut *metadata* versi secara signifikan menyelesaikan masalah tumpang-tindih informasi (*information overlap*) pada dokumen institusi akademik yang bersifat periodik.

4.5 Implementasi Sistem

Berdasarkan masukan dan ruang lingkup penelitian, implementasi sistem difokuskan pada pengembangan logika *backend* dan *pipeline* pemrosesan data, tanpa mengutamakan antarmuka grafis (UI). Interaksi pengguna berjalan secara tekstual, di mana fokus utama terletak pada bagaimana sistem memproses *query*, menyaring *metadata*, dan menghasilkan jawaban.

4.5.1 Komponen Pembangun Sistem

Sistem *chatbot version-aware* ini dibangun menggunakan ekosistem *open-source* yang berjalan secara lokal. Komponen utama yang diimplementasikan meliputi:

- **LangChain:** Digunakan sebagai *framework* utama yang menghubungkan *Large Language Model* (LLM), basis data vektor, dan komponen *prompting*.
- **ChromaDB:** Diimplementasikan sebagai *Vector Database* lokal untuk menyimpan *embeddings* dokumen beserta *metadata* (seperti ID dokumen dan tahun).
- **Ollama:** Digunakan sebagai mesin inferensi lokal untuk menjalankan model **Llama3:8b**. Penggunaan Ollama memastikan privasi data tetap terjaga karena seluruh proses *generation* terjadi di dalam perangkat tanpa memerlukan API eksternal.

4.5.2 Alur Pemrosesan Retrieval-Augmented Generation (RAG)

Implementasi pemrosesan pertanyaan pengguna mengikuti alur kerja yang terstruktur untuk memastikan jawaban yang dihasilkan akurat dan sesuai dengan versi yang diminta. Tahapan implementasi tersebut adalah sebagai berikut:

1. **Penerimaan Input (Query & Version):** Sistem menerima dua jenis input dari pengguna: (1) teks pertanyaan (*query*), dan (2) parameter versi dokumen yang ingin ditelusuri (misalnya: ”2024”).

2. **Pencarian Berbasis Metadata (Vector Search):** *Query* dari pengguna diubah menjadi vektor. Sebelum sistem melakukan pencarian *similarity* (kemiripan), sistem akan menerapkan *Pre-filtering* pada ChromaDB. Logika ini diimplementasikan dengan meneruskan parameter versi ke dalam argumen *filter* pada fungsi *retriever*, sehingga ruang pencarian hanya dibatasi pada *chunks* yang memiliki *metadata* tahun ”2024”.

3. **Injeksi Konteks (Prompt Construction):** Potongan teks (*chunks*) hasil pencarian yang lolos dari proses *filtering* kemudian digabungkan ke dalam *template prompt*. *Prompt* ini telah direkayasa (*prompt engineering*) dengan instruksi tegas agar LLM hanya menjawab berdasarkan konteks yang diberikan dan menolak menjawab jika informasi tidak ditemukan dalam konteks.

4. **Generasi Jawaban (Generation):** *Prompt* yang sudah berisi konteks spesifik versi tersebut dikirimkan ke model Llama3:8b via Ollama. LLM kemudian mensintesis kalimat jawaban yang natural dan mengembalikannya kepada pengguna.

4.5.3 Logika Filtering Version-Aware

Inti dari implementasi *version-aware* pada penelitian ini terletak pada manipulasi argumen pencarian pada modul *retriever*. Sistem tidak melakukan pencarian pada seluruh isi *database*, melainkan menggunakan struktur *dictionary* untuk memfilter atribut dokumen secara ketat.

Sebagai contoh logika, ketika pengguna ingin mencari aturan dari tahun 2024, sistem menerapkan konfigurasi filter `{"tahun": "2024"}`. Mekanisme ini memastikan bahwa sehebat apa pun tingkat kemiripan semantik (*semantic similarity*) dari dokumen kalender akademik tahun 2022 terhadap pertanyaan pengguna, dokumen tersebut akan diabaikan oleh sistem pada tahap awal penarikan data (*retrieval*). Pendekatan inilah yang secara signifikan meningkatkan nilai *Context Precision* pada hasil evaluasi, karena sistem kebal terhadap ambiguitas dokumen lintas tahun.

Universitas
Pertamina

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

5.2 Saran

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Universitas
Pertamina

DAFTAR PUSTAKA

Amazou, Y., Tayalati, F., dan Azmani, M. (2025). Accurate AI assistance in contract law using Retrieval-Augmented Generation to advance legal technology. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 16(2).

Anjaneyulu, D., Bhaskar, P., dan Yadav, S. H. (2025). RAG bot for random documents. *International Journal of Current Research and Techniques (IJCRT)*, 15(2):50722–50728.

Samudra, G., Zy, A. T., dan Ermanto, E. (2025). Implementasi Retrieval Augmented Generation (RAG) dalam perancangan chatbot kesehatan pencernaan. *JSAI: Journal Scientific and Applied Informatics*, 8(1):181–188.

Steininger, S., Kezer, S., Rief, J., Spicker, E., Preis, S., dan Fottner, J. (2025). ChAx: A RAG-based chatbot for CAx education. *Proceedings of the Design Society*, 5:921–930. ICED25.

Universitas
Pertamina

LAMPIRAN A

Kode Sumber Sistem Retrieval-Augmented Generation (RAG)

Pada lampiran ini, dilampirkan kode sumber dari sistem *Retrieval-Augmented Generation* (RAG) *version-aware* yang telah dikembangkan. Kode sumber dibagi menjadi beberapa modul utama berdasarkan fungsinya, yaitu pemrosesan dokumen, layanan *backend* API, antarmuka pengguna berbasis teks, serta skrip evaluasi sistem.

A.1 Pemrosesan Dokumen dan Basis Data Vektor

Bagian ini memuat skrip yang bertanggung jawab untuk mempersiapkan pangkalan pengetahuan (*knowledge base*). Skrip `ingest.py` berfungsi untuk mengekstraksi teks dari berbagai format dokumen (PDF, gambar, maupun teks biasa), membaginya menjadi potongan-potongan informasi (*chunks*) dengan tetap mempertahankan konteks lintas halaman, lalu mengubahnya menjadi representasi vektor (*embeddings*) menggunakan model `intfloat/multilingual-e5-base` sebelum disimpan ke dalam ChromaDB.

Skrip *Ingestion* Dokumen (`ingest.py`)

```python

def ingest_file(path, version, collection, model, manual_name=None):
    WINDOW_SIZE = 15
    STRIDE = 5

    i = 0
    while i < len(global_lines):
        window_items = global_lines[i : i + WINDOW_SIZE]
        chunk_text = "\n".join([item['text'] for item in window_items]).strip()
        start_page = window_items[0]['page']
        end_page = window_items[-1]['page']
        page_str = str(start_page) if start_page == end_page else f"{start_page}-{end_page}"

        if len(chunk_text) > 30:
            meta = {
                "doc_id": doc_uuid,
                "name": name,
                "version": version,
                "timestamp": ts,
```

```
                "page": page_str,
                "content": chunk_text
            }
            all_chunk_docs.append(chunk_text)
            all_chunk_metas.append(meta)
            all_chunk_ids.append(
                f"{doc_uuid}_{total_chunks}")
            total_chunks += 1
        i += STRIDE

    embeddings = model.encode(all_chunk_docs,
    ↪ normalize_embeddings=True)
    collection.add(
        documents=all_chunk_docs,
        embeddings=embeddings.tolist(),
        metadatas=all_chunk_metas,
        ids=all_chunk_ids
    )
```

Selanjutnya, skrip `inspect_db.py` digunakan sebagai alat bantu pemantauan untuk memverifikasi isi dari ChromaDB. Skrip ini membaca metadata dari pangkalan data dan menampilkan informasi ringkas mengenai daftar dokumen, versi dokumen, jumlah *chunk*, dan stempel waktu (*timestamp*).

Skrip Inspeksi Basis Data (`inspect_db.py`)

```

def main():
    client = chromadb.PersistentClient(path="./chroma_db")
    collection = client.get_collection("docs_versioned")
    all_data = collection.get(include=["metadatas"])

    # Struktur data untuk mengelompokkan chunk berdasarkan
    ↪ nama file
    files_summary = defaultdict(lambda: {
        'version': 'Unknown', 'timestamp': 'N/A',
        'doc_id': 'N/A', 'chunks': 0, 'pages': set()
    })

    for meta in all_data['metadatas']:
        fname = meta.get('filename') or meta.get('source')
        ↪ or "Tanpa Nama"
```

```
        clean_fname = fname.split('/')[-1].split('\\')[-1]

        files_summary[clean_fname]['chunks'] += 1

        # Ambil metadata umum pada temuan potongan teks (chunk) pertama
        if files_summary[clean_fname]['chunks'] == 1:
            files_summary[clean_fname]['version'] = meta.get('version', 'N/A')
            files_summary[clean_fname]['timestamp'] = meta.get('timestamp', 'N/A')

        # Hitung set halaman unik untuk memastikan kelengkapan parsing dokumen
        if 'page' in meta and meta['page'] is not None:
            files_summary
            [clean_fname]['pages'].add(meta['page'])

```

A.2 Layanan *Backend* API RAG

Skrip `app.py` merupakan inti dari sistem RAG yang dibangun menggunakan *framework* FastAPI. Modul ini bertugas menerima permintaan kueri dari pengguna, melakukan pencarian semantik (*similarity search*) pada ChromaDB dengan mempertimbangkan spesifikasi versi dokumen (*version-aware*), merakit *prompt* yang komprehensif, dan mengirimkannya ke model bahasa (LLM) Ollama untuk menghasilkan respons akhir berbahasa Indonesia.

Skrip *Backend* API (`app.py`)

```

@app.post("/query", response_model=QueryResponse)
async def handle_query(request: QueryRequest):
    if request.version == "latest":
        # Ambil sampel lebih besar untuk memfilter dokumen terbaru
        results = collection.query(
            query_embeddings=[q_emb_list], n_results=TOP_K_SEARCH,
            include=["metadatas", "distances", "documents"]
        )

        # Pengelompokan dokumen berdasarkan nama
```

```
        doc_groups = {}
        for (doc, meta, dist) in zip(results['documents'][0], results['metadatas'][0], results['distances'][0]):
            group_key = meta.get("name")
            if group_key not in doc_groups: doc_groups[group_key] = []
            doc_groups[group_key].append({"chunk": doc, "meta": meta, "dist": dist})

        final_candidates = []
        for name, items in doc_groups.items():
            # Filter: Hanya ambil potongan dokumen dengan timestamp terbaru
            latest_ts = max(item['meta'].get('timestamp', 0) for item in items)
            latest_chunks = [item for item in items if item['meta'].get('timestamp', 0) == latest_ts]
            final_candidates.extend(latest_chunks)

        final_candidates.sort(key=lambda x: x["dist"])
        top_chunks_with_meta = final_candidates[:request.top_k]

    else:
        results = collection.query(
            query_embeddings=[q_emb_list], n_results=request.top_k,
            where={"version": request.version},
            include=["metadatas", "distances", "documents"]
        )

    prompt = build_prompt(request.query, top_chunks_with_meta)
    answer = call_ollama_chat(prompt)

    return QueryResponse(answer=answer, provenance=format_provenance(top_chunks_with_meta))
```

A.3 Antarmuka Pengguna Berbasis Baris Perintah

Untuk memfasilitasi interaksi pengguna dengan layanan *backend*, dikembangkan sebuah antarmuka berbasis baris perintah (*Command Line Interface*) pada skrip `rag_cli_versionaware.py`. Antarmuka ini memungkinkan pengguna untuk mengirimkan kueri, mengganti versi dokumen target yang ingin dicari, serta menampilkan jawaban LLM beserta tabel sumber dokumen asal (*provenance*) dan skor kesamaannya.

Skrip Antarmuka CLI (`rag_cli_versionaware.py`)

```python

def ask_rag(query: str, version: str = "latest"):
    # Mengirimkan kueri beserta parameter versi ke endpoint RAG backend
    payload = {"query": query, "version": version, "top_k": 5}
    res = requests.post("http://localhost:8000/query", json=payload, timeout=120)
    return res.json() if res.status_code == 200 else None

def main():
    version = "latest" # Konfigurasi versi pencarian bawaan

    while True:
        cmd = Prompt.ask("[bold green] Pertanyaan / Perintah[/bold green]").strip()
        if cmd.lower() in ["exit", "quit"]: break

        if cmd.lower().startswith("pakai "):
            version = cmd.split("pakai ", 1)[1].strip()
            console.print(f" Versi diubah menjadi: {version}")
            continue

        result = ask_rag(cmd, version)
        if not result: continue

        answer = result.get("answer", "").strip()
        provenance = result.get("provenance", [])

        # Menampilkan jawaban utama dan tabel sumber rujukan
```

```
        console.print(Panel.fit(answer, title=f" Jawaban ({version})"))
        if provenance:
            show_provenance_table(provenance)

        log_chat(cmd, version, answer, provenance)
```

A.4 Evaluasi Kinerja Sistem

Tahap pengujian kinerja sistem RAG diimplementasikan dalam skrip `eval_ragas.py`. Skrip ini menggunakan kerangka kerja RAGAS (*Retrieval Augmented Generation Assessment*) untuk menghitung metrik evaluasi seperti *faithfulness*, *answer relevancy*, *context precision*, dan *context recall*. Hasil evaluasi ini kemudian diekspor ke dalam format *spreadsheet* untuk analisis lebih lanjut pada Bab 4.

Skrip Evaluasi Kinerja Sistem (`eval_ragas.py`)

```

def run_evaluation():
    judge_llm = ChatOllama(model="llama3:8b", temperature=0, format="json")
    judge_embeddings = HuggingFaceEmbeddings(
        model_name="intfloat/multilingual-e5-base")

    # Konversi data inferensi menjadi Dataset HuggingFace
    dataset = Dataset.from_dict(data_dict)
    results = evaluate(
        dataset=dataset,
        metrics=[
            faithfulness,
            answer_relevancy,
            context_precision,
            context_recall,
        ],
        llm=judge_llm,
        embeddings=judge_embeddings,
        run_config=RunConfig(max_workers=1, timeout=300)
    )

    # Simpan laporan evaluasi ke spreadsheet
    df = results.to_pandas()
    df.to_excel("laporan_evaluasi_ragas.xlsx", index=False)
```

LAMPIRAN B

Judul Lampiran 2

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.